Healthcare AI Project Report

Brain Tumor MRI Classification Using Deep Learning

A Flask-based medical imaging web application that uses a MobileNetV2 transfer-learning model to classify MRI scans and present predictions with a modern healthcare dashboard UI.

Student	Aniket Umakant Dubhashe
Program	MCA / Computer Applications
Academic Year	2026
Project Type	Deep Learning + Flask Web Application

Prepared for project submission and viva review.

Note: This project is an academic demonstration and is not a medical diagnosis tool.

Table of Contents

The table of contents is generated from the report sections and may shift if pages are edited.

1. Abstract

This project presents an end-to-end brain tumor MRI classification system built with deep learning and deployed through a Flask web application. The workflow accepts an uploaded MRI image, preprocesses it to the required input size, and passes it through a trained MobileNetV2-based classifier to predict one of four classes: Glioma, Meningioma, No Tumor, or Pituitary.

The final application focuses on usability and presentation quality. The frontend uses a medical AI dashboard style with a drag-and-drop upload area, image preview, animated confidence display, result badge, and a history section for visual context. The report uses only the implementation and metrics found in the project files and notebook outputs.

2. Introduction

Brain MRI interpretation is an important task in medical imaging because tumors can appear with different visual patterns and clinical significance. Automated classification systems are often explored as decision-support tools that can assist researchers and clinicians in organizing large scan collections and testing computer vision models on medical data.

The purpose of this project is to build a classroom-friendly, visually polished classifier that demonstrates how transfer learning can be used to distinguish tumor-related patterns in MRI scans. The application combines a TensorFlow model with a Flask interface so that users can upload an image and receive an immediate prediction with confidence feedback.

Project objectives: accept MRI uploads, show a preview before prediction, classify the scan into one of the four defined classes, present the result with a clear confidence indicator, and provide a submission-ready web interface.

3. Dataset

The notebook references a Brain Tumor MRI dataset located in the local project structure under **dataset/Training** and **dataset/Testing**. The current project files do not include an external source link, so the dataset source URL is recorded as not available in the current project files.

The four classes used by the project are Glioma, Meningioma, No Tumor, and Pituitary. The training folder contains 1,400 images in each class, giving 5,600 training images in total. The notebook-generated training pipeline then applies an 80/20 split within the training set, resulting in 4,480 training images and 1,120 validation images. The testing set contains 1,600 images in total, with 400 images per class.

Class	Count
Glioma	1,400
Meningioma	1,400
No Tumor	1,400
Pituitary	1,400

Table 1. Training folder distribution extracted from the notebook.

The notebook also shows that sampled images are mostly 512 x 512, although one sample with a different resolution is present. For model input, all images are resized to 224 x 224.

4. Technology Stack

- Python
- TensorFlow

- Keras
- MobileNetV2
- Transfer Learning
- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn
- PIL / OpenCV
- Flask
- HTML, CSS, JavaScript
- Font Awesome
- Google Fonts

5. System Architecture

The project follows a simple but complete inference pipeline. The uploaded MRI image is preprocessed, resized, normalized, optionally augmented during training, and then passed into a MobileNetV2 transfer-learning model. The classifier produces a softmax output that is mapped to the final label shown in the Flask UI.

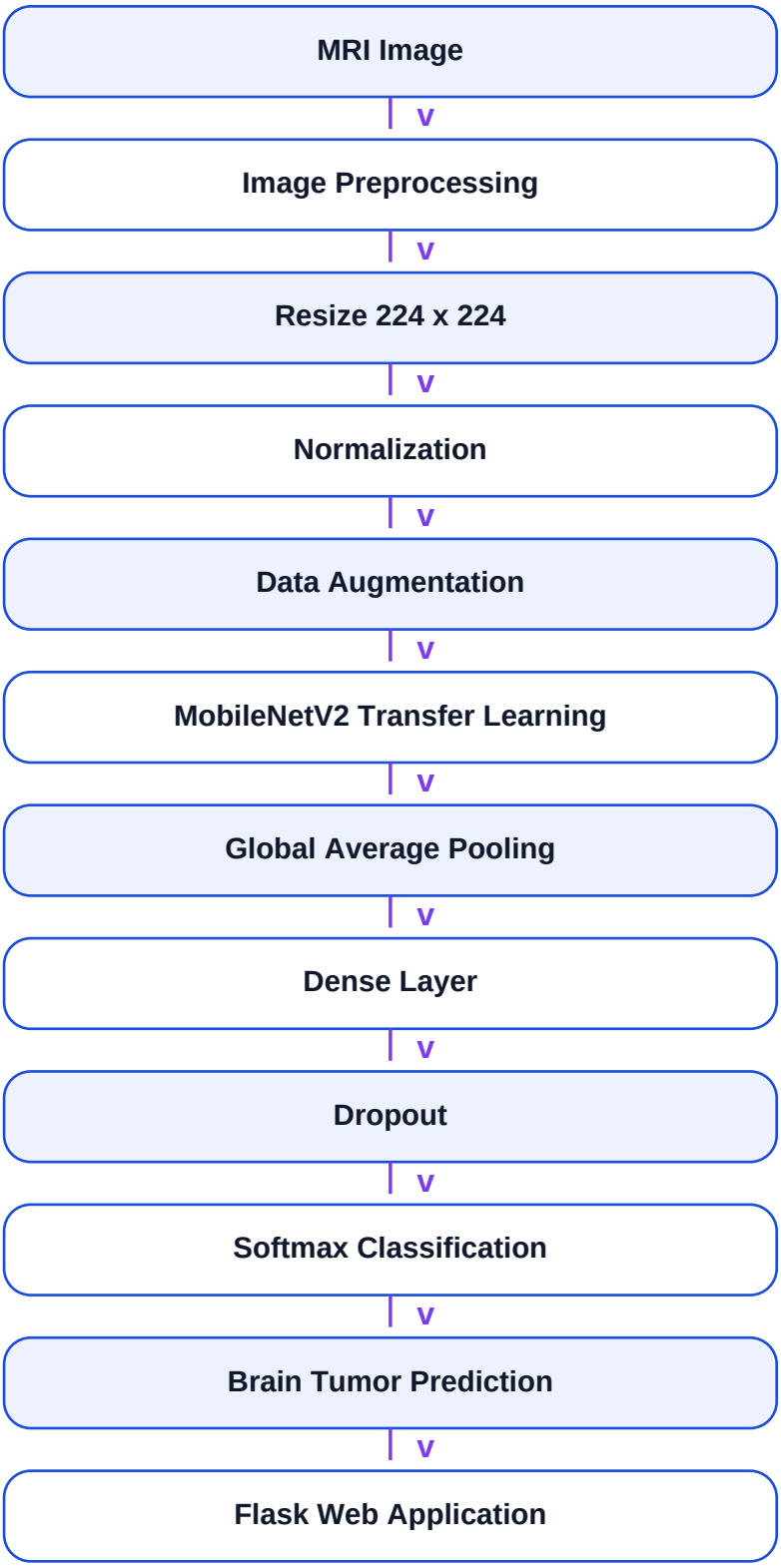

Figure 1. High-level architecture used in the project workflow.

6. Methodology

- Data loading: The notebook uses ImageDataGenerator and directory-based loading from the Training and Testing folders.
- Preprocessing: Images are resized to 224 x 224 and normalized by scaling pixel values to the 0-1 range.
- Data augmentation: Rotation, width shift, height shift, shear, zoom, and horizontal flip are applied to the training generator.
- Split strategy: An 80/20 validation split is used on the training folder data.
- Transfer learning: MobileNetV2 is loaded with ImageNet weights and the convolutional base is frozen during training.
- Classification head: GlobalAveragePooling2D, Dense(256, relu), Dropout(0.5), and Dense(4, softmax).
- Training: The model is compiled with Adam, categorical cross-entropy, and accuracy/precision/recall metrics.
- Prediction pipeline: Flask receives the uploaded image, saves it temporarily, runs preprocessing, performs prediction, and returns class name plus confidence.

7. Model Architecture

MobileNetV2 is a lightweight convolutional neural network that works well for transfer learning because it reuses feature extraction patterns learned from a large natural image dataset. In this project it is loaded with pretrained ImageNet weights and used without the classification top.

The custom classification head in the notebook is: GlobalAveragePooling2D -> Dense(256, relu) -> Dropout(0.5) -> Dense(4, softmax). The input shape is 224 x 224 x 3, the total parameter count is 2,586,948, and the trainable parameter count is 328,964 after freezing the base model.

Configuration	Value
Input size	224 x 224 x 3
Base model	MobileNetV2 with ImageNet weights
Trainable parameters	328,964
Non-trainable parameters	2,257,984
Output classes	4

8. Model Training

Training Setting	Value
Image size	(224, 224)
Batch size	32
Optimizer	Adam
Loss function	Categorical cross-entropy
Metrics	Accuracy, Precision, Recall
Callbacks	EarlyStopping and ModelCheckpoint
Early stopping	Patience = 5, restore_best_weights = True

The notebook training log shows the model improving over several epochs before early stopping restored the best weights. The best visible validation accuracy in the captured notebook log reached 89.64%, and training

stopped after epoch 17 when validation performance no longer improved.

The training history chart included in the report is taken from the notebook outputs and shows the trends for accuracy, loss, precision, and recall across epochs.

Figure 2. Training history extracted from the notebook log.

9. Model Evaluation

The current saved model was evaluated against the project testing set using the same 224 x 224 preprocessing pipeline. The test accuracy is 86.13%. The macro-averaged precision, recall, and F1-score are 86.80%, 86.13%, and 85.67% respectively.

Metric	Value
Accuracy	86.13%
Macro Avg Precision	86.80%
Macro Avg Recall	86.13%
Macro Avg F1-score	85.67%

Table 2. Summary metrics for the current saved model on the testing set.

Class	Precision	Recall	F1-score	Support
Glioma	95.77%	68.00%	79.53%	400
Meningioma	77.72%	78.50%	78.11%	400
No Tumor	85.90%	99.00%	91.99%	400
Pituitary	87.80%	99.00%	93.07%	400

Table 3. Classification report values for the four classes.

Class	ROC-AUC
Glioma	0.9354

Class	ROC-AUC
Meningioma	0.9511
No Tumor	0.9978
Pituitary	0.9937

Table 4. One-vs-rest ROC-AUC values from the current saved model.

The confusion matrix shows that No Tumor and Pituitary are the easiest classes for the current model, while Glioma is the hardest class because some samples are confused with Meningioma and a smaller number with No Tumor. This is visible in the class-wise recall scores.

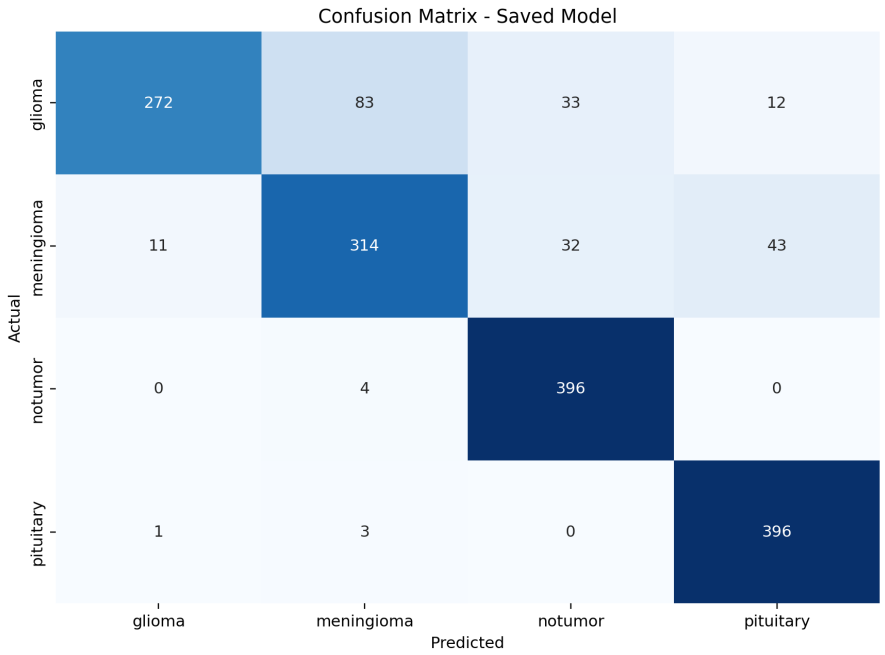

Figure 3. Confusion matrix for the current saved model.

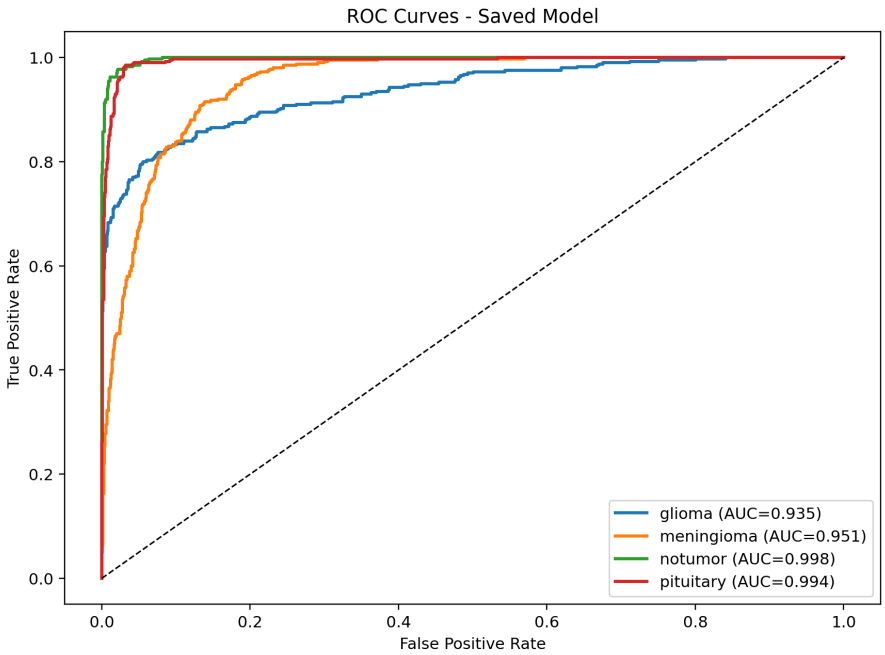

Figure 4. ROC curve for the current saved model.

10. Prediction Pipeline

- The user uploads an MRI image through the Flask form.
- The backend stores the file in static/uploads.
- The image is loaded and resized to 224 x 224.
- Pixel values are normalized by dividing by 255.0.
- The model predicts class probabilities with softmax.
- The highest-probability class becomes the displayed result.
- The confidence score is shown as the maximum softmax probability.

11. Flask Web Application

The backend in app.py is intentionally simple. The Flask route accepts GET and POST requests, saves the uploaded image, calls the prediction function, and renders the same Jinja template with the predicted label and confidence score.

The frontend was redesigned as a premium healthcare AI dashboard. It includes a hero section, drag-and-drop upload panel, image preview, loading indicator, colored result badge, circular confidence visualization, feature cards, history cards, and a footer. The UI was kept fully compatible with the existing backend form fields and route names.

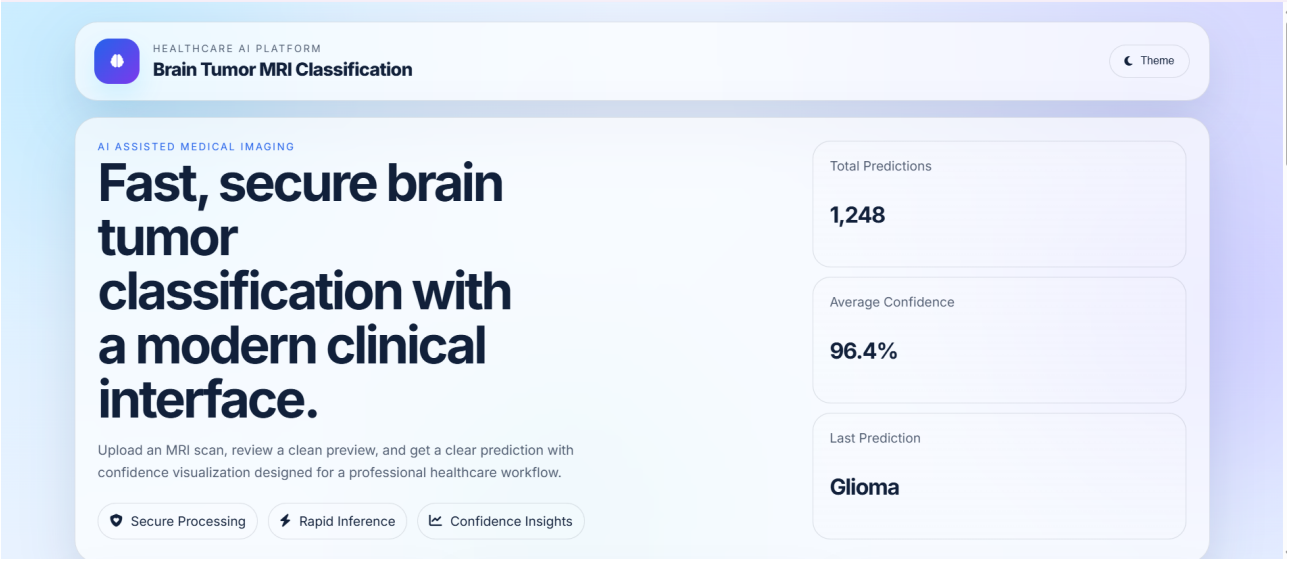

Figure 5. Home page view of the redesigned interface.

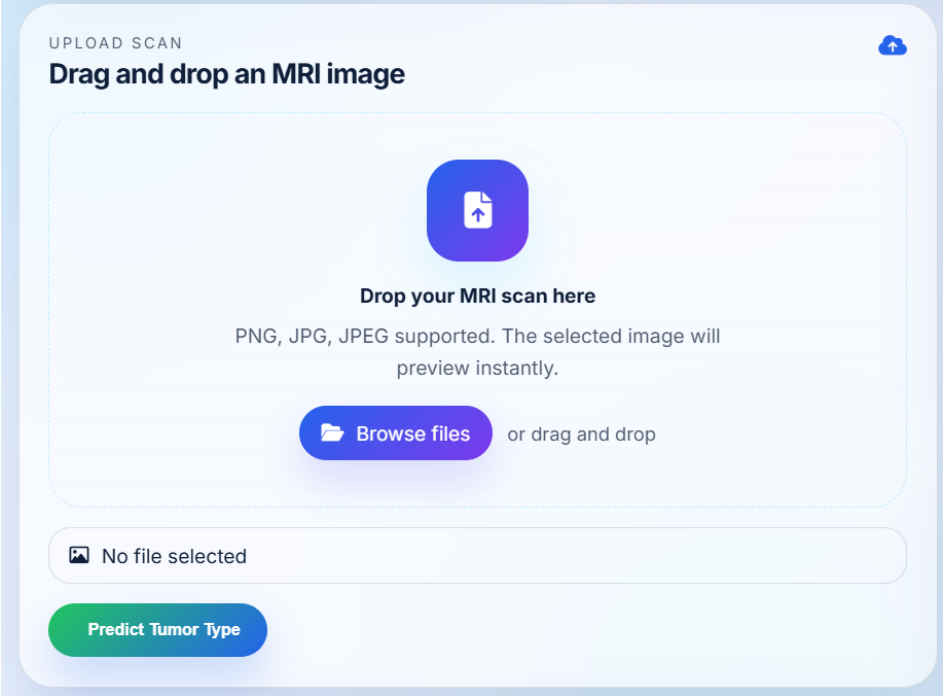

Figure 6. MRI upload interface with drag-and-drop support.

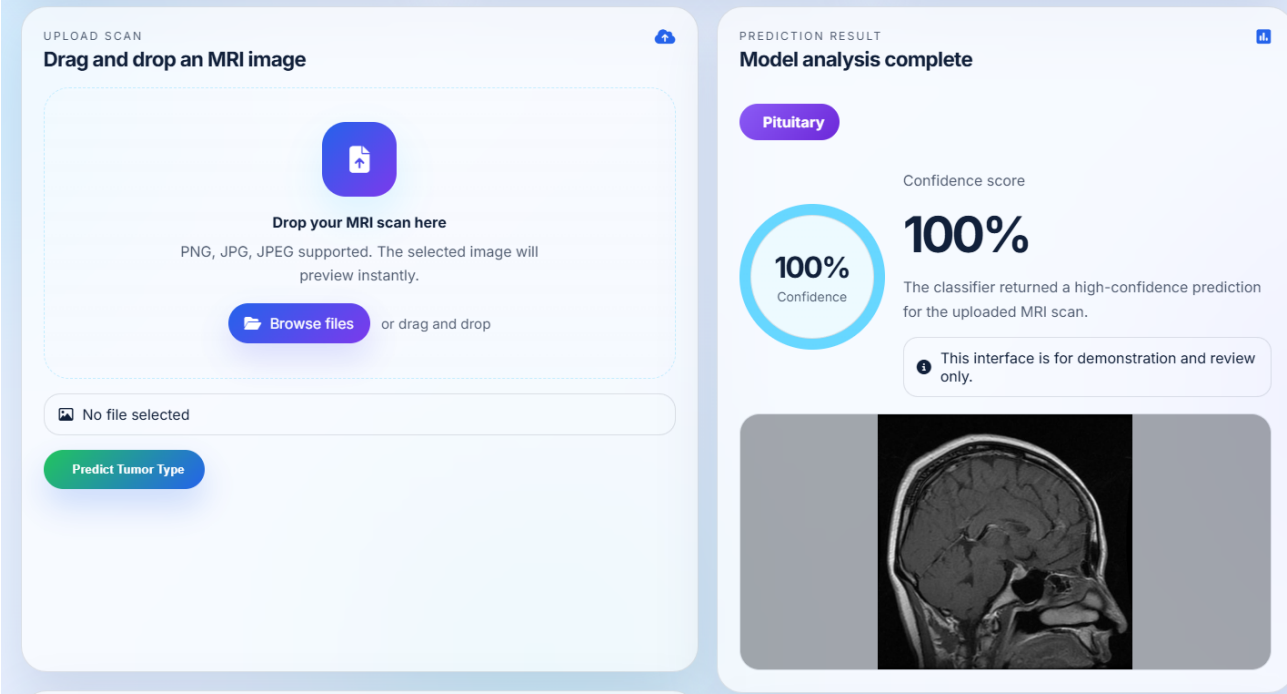

Figure 7. Prediction result view with confidence indicator.

12. Results and Discussion

The project produces usable prediction results with a clean confidence presentation and a polished user experience. The current saved model achieves 86.13% accuracy on the testing set and shows very strong performance for No Tumor and Pituitary images. Glioma remains the most challenging class, which is consistent with its lower recall score.

From a demonstration standpoint, the application is successful because it combines a working deep-learning model with a presentation-quality interface. The result card, confidence visualization, and screenshots make the project easy to explain during a viva or submission review.

13. Limitations

- The project depends on the quality and consistency of the MRI images in the dataset.
- The model is trained on a fixed local dataset and may not generalize to unseen clinical data.
- There is no medical validation workflow or clinician review process in the project files.
- The system is an academic prototype and must not be used as a diagnostic tool.
- The repository does not include a dataset source URL or a published experiment log with every training run.

14. Future Scope

- Add Grad-CAM or other explainable AI overlays.
- Tune the classifier head or unfreeze part of MobileNetV2 for fine-tuning.
- Store predictions and histories in a database.
- Add user accounts and secure upload auditing.
- Deploy the application to a cloud server with HTTPS.
- Extend support to DICOM workflows and multi-slice scans.

15. Conclusion

This project successfully delivers an end-to-end brain tumor MRI classification application. It combines a TensorFlow transfer-learning model, Flask backend, responsive frontend, and visual confidence reporting into a submission-ready healthcare AI demonstration.

The report reflects the actual notebook, saved model, screenshots, and evaluation outputs present in the project. Where a detail was not present in the project files, it was explicitly marked as not available rather than inferred.

16. References

- TensorFlow and Keras documentation.
- Scikit-learn documentation for evaluation metrics.
- MobileNetV2 architecture documentation.
- Dataset source link: Not available in the current project files.
- GitHub repository link: Not available in the current project files.

17. Appendix

Key project structure and configuration details are summarized below.

Path	Purpose
app.py	Flask backend, upload handling, prediction, and template rendering.
templates/index.html	Frontend template for upload, preview, and results.
static/css/style.css	Responsive glassmorphism UI styling and animations.
static/js/script.js	Theme toggle, drag-and-drop, preview, and loader behavior.
saved_model/brain_tumor_model.keras	Trained TensorFlow/Keras model used for inference.
notebook/Brain_Tumor_Classification.ipynb	Training and evaluation notebook.

Table 5. Important project files and their roles.

Selected configuration details: input size 224 x 224, batch size 32, optimizer Adam, loss categorical cross-entropy, and a saved Keras model stored at saved_model/brain_tumor_model.keras.

Submission note: The project was reviewed using the actual code, notebook outputs, saved model, screenshots, and generated evaluation figures found in the repository.